STRATOS

A Game-Theoretic System for Strategic University Project Team Formation

Team Work Allocation and Execution Plan

Game Theory (CSE3145) | Manipal University Jaipur

Planning phase, v1.0 | 25 October 2026

Member	Role	Tools
Aryan Rajguru	Game Theory engine and integration lead	Claude Code
Harsh	Frontend and interactive application	Claude Code
Ayush	Quality assurance, documentation and presentation	ChatGPT + shared GitHub repository

Status: planning deliverable. No implementation exists yet; every test is planned, none executed, and no result is claimed.

Contents

1. Project title, team and plan at a glance . . . 3
2. Assignment requirements and evaluation criteria . . . 4
3. Recommended project scope and features . . . 5
4. Mathematical model overview . . . 5
5. Architecture and data flow . . . 7
6. Repository structure and file ownership . . . 8
7. Work packages . . . 9
8. Dependencies and interface contract (engine <-> app) . . . 14
9. Hour-by-hour execution plan . . . 16
10. Priorities and acceptance criteria . . . 18
11. Test plan . . . 18
12. Report and presentation outlines . . . 20
13. Demo plan and viva preparation . . . 21
14. Risks, fallback strategy and final checklist . . . 23

Reading guide: **P0** required for the MVP, **P1** strong enhancement, **P2** optional. Times are written T+hours:minutes from kickoff. Task IDs: A = Aryan, F = Frontend (Harsh), Q = Quality/docs (Ayush); test IDs: E engine, I integration, S UI smoke, O oracle, M manual.

1. Project title, team and plan at a glance

Member	Role	Tools
Aryan Rajguru	Game Theory engine and integration lead	Claude Code
Harsh	Frontend and interactive application	Claude Code
Ayush	Quality assurance, documentation and presentation	ChatGPT + shared GitHub repository

What STRATOS is. An interactive Python/Streamlit application in which eight fictional students choose among three university projects. Each student's payoff depends on how much they like the project, how well their skills complement their teammates, and how crowded the team is. The application evaluates any assignment, lists every profitable unilateral deviation, simulates best-response dynamics, and **verifies** stability by re-checking the Nash condition. For this small game it also enumerates all 3^8 = 6,561 assignments to list every pure Nash equilibrium and compare it with the welfare-maximising assignment.

Plan in one sentence. Aryan builds and proves the engine, Harsh builds the Streamlit app against a frozen interface, Ayush independently re-implements and hand-checks the results, tests the real app, and turns actual outputs into the report, slides and viva preparation. Everything except the engine interface is replaceable; correctness and an end-to-end working demo come first.

1.1 What was verified in the workspace (Phase A)

Item	Finding	Status
Repository `D:\STRATOS`	Empty directory: no code, no documents, no existing instructions.	Verified
Git	Not a Git repository. `git init`, the GitHub remote and branches are steps for the team (Section 6); none were performed.	Verified
`Groups.xlsx` and course handout	Not found in the workspace, Downloads, Desktop or Documents (searched to depth 2). The assigned title, group number and any faculty-specific rules are therefore **unverified**.	Not found
Python toolchain (Aryan's machine)	Python 3.11.9, Streamlit 1.64.0, pytest 9.1.1, pandas 3.0.5, numpy 2.4.2, openpyxl present. Plotly is not installed and is not planned; `st.bar_chart` / `st.line_chart` are enough.	Verified
Harsh's and Ayush's machines	Not inspected. `requirements.txt` will use minimum versions, and Ayush verifies setup from a clean clone.	Unverified
PDF tooling	`reportlab` 5.1.0 was installed with pip solely to generate this PDF; it is **not** a project dependency.	Verified

1.2 Explicit assumptions

ID	Assumption	Owner who resolves it
A-1	The assigned topic is 'Strategic Behavior in University Course/Project Team Formation', as stated in the task brief. The exact title/group row in `Groups.xlsx` has not been seen.	Ayush (task Q1, first 30 minutes)
A-2	Marks are 10 in total across implementation, functionality, understanding, documentation and presentation. No per-criterion split has been published to us.	Ayush (Q1)
A-3	Report and slide file formats are not specified. Plan: report as `.docx` and `.pdf`, presentation as `.pptx`.	Ayush (Q1)
A-4	Ayush can run Python 3.11+ locally. If not, the fallback in Section 14 applies (screen-share testing, ChatGPT code interpreter, Excel oracle).	Ayush (at T+0:30)
A-5	All student names, skills and preferences are **fictional and synthetic**. Utility weights are modelling assumptions, not measurements, and the report must say so.	Aryan
A-6	'One day' means 24 hours of elapsed time from kickoff T+0, including a 1-hour break. Times below are written T+n.	All
A-7	No database, authentication, API, machine-learning model or real student data is needed or planned.	All

2. Assignment requirements and evaluation criteria

2.1 Deliverables (from the assignment text)

#	Deliverable	Our concrete artefact	Owner
1	Working implementation (Python or other)	`app.py` + `game_engine.py` + `sample_data.py`, run with `streamlit run app.py`	Aryan + Harsh
2	Project report: problem statement, objectives, methodology, implementation, results, screenshots, conclusion	`docs/STRATOS_Report.docx` and `.pdf`, built from `docs/REPORT.md`	Ayush
3	PowerPoint presentation: project, objectives, implementation, results, applications	`presentation/STRATOS.pptx` (12 slides)	Ayush
4	Project title and group per `Groups.xlsx`	Verified and recorded in `docs/ASSIGNMENT_CHECK.md`	Ayush

2.2 Evaluation criteria and the evidence we will present

Criterion (10 marks total)	What the evaluator can see	Where our evidence comes from
Implementation	Clean engine, separated from UI; tests; reproducible setup	`game_engine.py`, `tests/`, `README.md`, `requirements.txt`
Functionality	Application runs; every advertised feature works; invalid input handled	Live demo, `docs/TEST_LOG.md` executed results, screenshots
Understanding of the project	Each member explains model, algorithm and limits unaided	`docs/MODEL.md`, `docs/VIVA.md`, rehearsal at T+20h
Documentation	Report covers all required sections with real screenshots and outputs	`docs/STRATOS_Report.*`, `docs/results/`, `screenshots/`
Presentation	Clear story, readable slides, confident reproducible demo	`presentation/STRATOS.pptx`, `docs/DEMO_SCRIPT.md`

2.3 Game Theory topics: what we use and what we deliberately leave out

Course topic	Decision	Reason
Strategies, costs and payoffs	**Use**	Core of the model: project choice, utility with teammate benefits and crowding cost
Nash equilibrium (pure)	**Use**	Central result: define, check by exhaustive unilateral-deviation test, enumerate
Best response / best-response dynamics	**Use**	Simulates strategic adjustment; terminates because the game is a potential game
Potential / congestion-style games	**Use**	Gives a proof that a pure equilibrium exists and that dynamics terminate
Pareto optimality, efficiency	**Use (P1)**	Cheap to compute on 6,561 profiles: social welfare, welfare gap, Pareto flag of each equilibrium
Static vs dynamic games	**Discuss**	The model is a static (simultaneous-move, complete-information) game. Best-response dynamics is an adjustment process, not an extensive-form game; we say so explicitly
Mixed strategies	Do not implement	A pure equilibrium is guaranteed here; mixed strategies are only used in the no-equilibrium test game (matching pennies)
Dominance	Optional note	Not needed for the demonstration
Correlated equilibrium, repeated games, bargaining, VCG, auctions	Future work only	Would not improve correctness in one day; mentioned as extensions (for example a VCG-style allocation as a mechanism-design remedy)

3. Recommended project scope and features

The scope is the smallest coherent set that produces a convincing, mathematically sound demonstration. Optional work starts only after the minimum viable product (MVP) passes the checkpoint at T+7:30.

Tier	Feature	Priority
MVP-1	Fixed fictional scenario: 8 students (skills), 3 projects, preference matrix, compatibility matrix, crowding parameter	P0
MVP-2	User builds an assignment (one project per student) or generates a seeded random one	P0
MVP-3	Per-student utility, team composition, team sizes, social welfare and potential for any assignment	P0
MVP-4	Equilibrium check: states 'stable' or lists every profitable unilateral deviation (student, from, to, gain)	P0
MVP-5	Best-response dynamics from a chosen start, with step-by-step trace and an independent final `is_nash` verification	P0
MVP-6	Adjustable crowding parameter gamma; results recompute	P0
MVP-7	Input validation and friendly errors; no tracebacks in the UI	P0
MVP-8	Automated engine tests, independent re-implementation and hand-computed oracle agree	P0
P1-1	Exhaustive enumeration of all 6,561 profiles: list of all pure Nash equilibria, welfare-optimal profile, welfare gap, price of anarchy / stability (when welfare is positive), Pareto flag	P1
P1-2	Comparison view: initial vs best-response result vs best equilibrium vs welfare optimum	P1
P1-3	CLI report `python game_engine.py --report` (JSON) used as evidence in the report	P0 basic / P1 extras
P1-4	Demo-mode button that loads the exact start assignment used in `docs/DEMO_SCRIPT.md`	P2
P2-1	Sensitivity sweep over gamma; random-start convergence statistics; editable preferences (`st.data_editor`); JSON/CSV download; UI polish	P2

Non-goals: database, login, REST API, machine learning, real student data, mixed-strategy solver, scaling beyond a classroom-size demo, and any claim that the equilibrium is 'the best' team formation.

4. Mathematical model overview

Full statement and proof sketch live in `docs/MODEL.md` (task A1). This section fixes the model so all three members build against the same definitions. All indices are 0-based in code and in the interface.

Element	Definition
Players	Students N = {0, ..., n-1}, with n = 8 in the default scenario.
Strategies	Each student picks exactly one project from M = {0, ..., m-1}, with m = 3. Strategy of student i: `s_i`.
Assignment (profile)	`s = (s_0, ..., s_{n-1})`, a list of n integers in 0..m-1. There are m^n = 3^8 = 6,561 profiles. Team of project t: `T_t(s) = {i : s_i = t}`, size `n_t(s)`. Empty teams are allowed.
Parameters	`p[i][t]`: preference of student i for project t (integer 0..10). `w[i][j]`: teammate compatibility, **symmetric** (`w[i][j] = w[j][i]`) with `w[i][i] = 0`. `c(k)`: crowding cost of a team of size k, depending only on k.
Skill-based compatibility	Each student has a skill set S_i from {Programming, Analysis, Design, Communication}. Default `w[i][j] = lambda * |S_i symmetric-difference S_j| - beta`, symmetric by construction (complementary skills help, near-identical students help less). Aryan fixes lambda and beta in `sample_data.py`; a directly typed matrix is also allowed.
Crowding cost	`c(k) = gamma * (k - 1)^2` with gamma >= 0, so c(1) = 0, c is convex and non-decreasing. A soft penalty is used on purpose: a hard size cap would make some deviations infeasible and complicate the equilibrium definition.

4.1 Utility (payoff) function

```
u_i(s) = p[i][s_i]  +  SUM over j != i with s_j = s_i of w[i][j]  -  c( n_{s_i}(s) )
         preference     teammate compatibility                     crowding cost
```

4.2 Best response, deviations and Nash equilibrium

- **Unilateral deviation:** student i switches to project t while everyone else keeps their choice, giving profile `(t, s_-i)`. Gain = `u_i(t, s_-i) - u_i(s)`.
- **Best response:** `BR_i(s_-i)` = the project(s) maximising `u_i(t, s_-i)`. Ties are broken by lowest project index. A student moves only for a **strictly** positive gain.
- **Pure Nash equilibrium (weak form):** `s*` is a pure Nash equilibrium iff for every student i and every project t, `u_i(s*) >= u_i(t, s*_-i)`. Equivalently: no student has a profitable unilateral deviation. The check costs n x (m-1) = 16 deviation evaluations per profile.
- **Numeric safety:** use integer inputs; the engine compares with tolerance EPS = 1e-9 so floats cannot create phantom deviations.

4.3 Why an equilibrium exists (exact potential game)

```
Phi(s) = SUM_i p[i][s_i]  +  SUM over pairs i<j with s_i = s_j of w[i][j]  -  SUM_t SUM_{k=1..n_t(s)} c(k)

For any unilateral deviation of student i from s to s':   u_i(s') - u_i(s) = Phi(s') - Phi(s)
```

- Each strictly profitable move raises Phi by exactly the mover's gain, so best-response dynamics can never cycle; there are finitely many profiles, so it terminates at a profile with no profitable deviation, i.e. a pure Nash equilibrium. A maximiser of Phi is also a pure equilibrium, so one always exists.
- The guarantee needs **symmetric** w and a crowding cost that depends only on team size. An asymmetric w or a non-pairwise 'team covers a skill' bonus can break it; the code still checks the equilibrium condition directly and never relies on the proof.
- Phi is **not** social welfare, and an equilibrium is **not** necessarily welfare-optimal or Pareto-optimal. The application reports all three notions separately.

4.4 Efficiency measures

```
W(s)  = SUM_i u_i(s) = SUM_i p[i][s_i] + 2 * SUM over pairs same team of w[i][j] - SUM_t n_t * c(n_t)
Welfare optimum: s_W = argmax W(s) over all 6,561 profiles (exhaustive)
Welfare gap of an equilibrium s*: W(s_W) - W(s*)     (always reported)
Price of anarchy  = W(s_W) / min over equilibria of W      (only when all W > 0)
Price of stability = W(s_W) / max over equilibria of W     (only when all W > 0)
Pareto-dominated: exists s' with u_i(s') >= u_i(s) for all i and > for some i
```

4.5 Algorithms and complexity

```
best_response_dynamics(scenario, start, order, seed, max_rounds):
    s = start;  steps = []
    repeat up to max_rounds sweeps:
        changed = False
        for i in order of this sweep (ascending index, or a seeded random permutation):
            t* = lowest-index argmax_t u_i(t, s_-i)
            if u_i(t*, s_-i) > u_i(s) + EPS:  s_i = t*;  record step;  changed = True
        if not changed: converged = True; stop
    return converged, final profile, steps;   callers re-verify with is_nash(final)
```

Operation	Cost	Note
Evaluate one utility	O(n)	recount teammates of the chosen project
Best response of one student	O(n m)	evaluate m alternatives
One sweep of dynamics	O(n^2 m)	number of sweeps is finite because Phi strictly increases; we report the **measured** count, no theoretical speed claim
`is_nash` on one profile	O(n^2 m)	all n (m-1) deviations

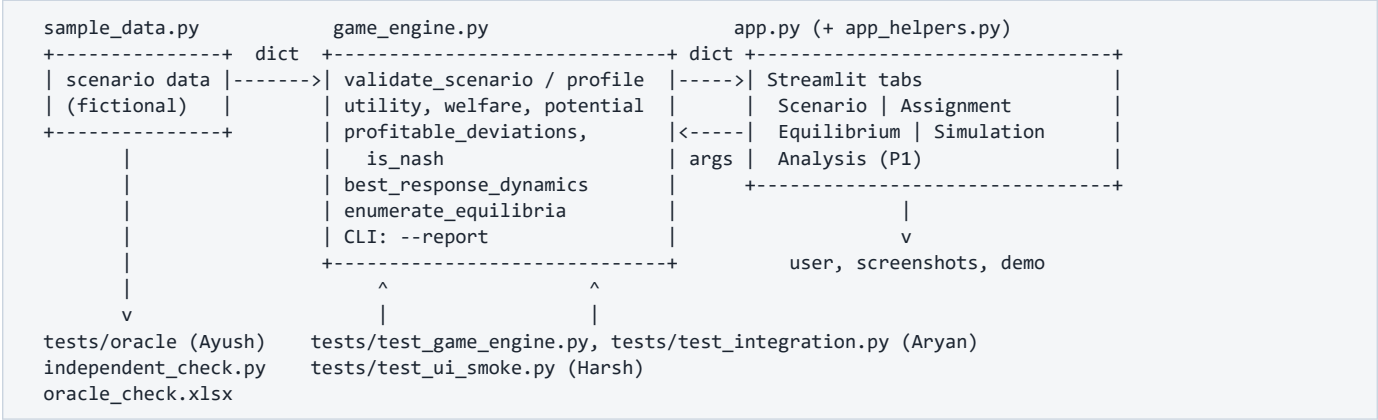
Operation	Cost	Note
Exhaustive enumeration	O(m^n n^2 m)	about 1.3 million basic operations for n = 8, m = 3; actual runtime to be measured and recorded in `docs/results/timing.txt`. Does not scale beyond roughly a dozen students; dynamics does

4.6 How the implementation verifies its own output

1. `is_nash` is a separate function that checks every unilateral deviation; the dynamics' 'converged' flag is never trusted on its own.
2. Property tests: exact potential identity on many random (profile, student, new project) triples; welfare equals the sum of utilities; deterministic given a seed.
3. Hand-computed expectations (the 2-student example in Section 8) and a known no-equilibrium game (matching pennies) through a generic checker.
4. Exhaustive enumeration cross-checked against 200 seeded best-response runs: every endpoint must be in the enumerated equilibrium set.
5. Ayush's independent re-implementation (written from `MODEL.md` only) and Excel oracle must produce the same equilibrium set and utilities.
6. UI numbers must equal engine numbers and CLI report numbers on at least 3 assignments.

Honesty rules for report, slides and demo: never call an equilibrium 'optimal'; say 'stable against unilateral deviations'; report every equilibrium found, not only one; state that the potential argument guarantees existence for THIS model only; disclose that data and weights are synthetic and were chosen to make the behaviour visible.

5. Architecture and data flow



- **One-way dependency:** `app.py` imports `game_engine`; `game_engine` imports nothing from the UI and never prints or touches Streamlit.
- **Data flow:** scenario dict + profile list in; plain JSON-serialisable dict/list out (no NumPy types, no tuples, no custom classes).
- **State:** `st.session_state` holds the scenario parameters, the current profile and the last simulation; enumeration is cached with `st.cache_data` keyed on the scenario.
- **Errors:** the engine raises `ScenarioError` / `ProfileError` (both subclasses of `StratosError`, a `ValueError`); the UI catches `StratosError` and shows `st.error`.

Choice	Decision	Why
Language / UI	Python 3.11, Streamlit	Already installed on Aryan's machine; fastest route to an interactive demo; one `streamlit run` command
Data handling	Plain lists and dicts; pandas only for display tables	Small data; avoids hidden type problems in tests
Charts	`st.bar_chart`, `st.line_chart`	Plotly is not installed and adds no required capability
Testing	pytest + Streamlit `AppTest` for UI smoke tests	Both available locally; no browser automation needed

Choice	Decision	Why
Dependencies	`streamlit>=1.30`, `pandas`, `pytest`	Minimal; Section 6 lists the file

6. Repository structure and file ownership

The repository was empty, so this structure is proposed from scratch. It follows the suggested layout with a few justified additions (`app_helpers.py`, `docs/MODEL.md`, `docs/INTERFACE.md`, `docs/results/`, `tests/oracle/`). Every file has exactly one owner; other members request changes through the owner or a pull request.

```
STRATOS/
  app.py                          Harsh   Streamlit entry point
  app_helpers.py                  Harsh   DataFrame builders, error boundary
  game_engine.py                  Aryan   engine + CLI (--report)
  sample_data.py                  Aryan   fictional scenario, tiny 2x2 scenario
  requirements.txt                Aryan
  .gitignore                      Aryan
  README.md                       Ayush   setup, run, test, repo map
  tests/
    test_game_engine.py           Aryan   E-01 .. E-14
    test_integration.py           Aryan   I-01 .. I-05
    test_ui_smoke.py              Harsh   S-01 .. S-04 (Streamlit AppTest)
    oracle/
      independent_check.py        Ayush   independent brute force, does not import game_engine
      independent_output.txt      Ayush   captured output
      oracle_check.xlsx           Ayush   hand-computed utilities with formulas
  docs/
    MODEL.md                      Aryan   formal model and proof sketch
    INTERFACE.md                  Aryan   engine <-> UI contract
    results/                      Aryan   engine_report.json, pytest_output.txt, timing.txt
    ASSIGNMENT_CHECK.md           Ayush   Groups.xlsx / handout findings
    TEST_PLAN.md, TEST_LOG.md     Ayush   planned vs executed tests
    DEMO_SCRIPT.md                Ayush   exact demo steps (reviewed by Aryan, Harsh)
    REPORT.md, STRATOS_Report.*   Ayush   report source and exports
    VIVA.md                       Ayush   questions and answers
    TEAM_WORK_ALLOCATION.md       Aryan   this plan (generated)
    plan_content.py, build_plan.py  Aryan  plan generator (optional to keep)
  presentation/
    SLIDES.md, STRATOS.pptx       Ayush
  screenshots/                    Ayush   S1..S8 from the frozen app, plus INDEX.md
  STRATOS_Team_Work_Allocation_and_Execution_Plan.pdf    Aryan
```

6.1 Branching and merge rules (proposed; nothing has been created)

Branch	Owner	Contains	Merges into main when
`main`	Aryan (integrator)	Always runnable and tested	n/a: only via pull request or agreed merge
`feat/engine`	Aryan	engine, sample data, engine tests, MODEL/INTERFACE	`pytest tests/test_game_engine.py` green
`feat/ui`	Harsh	`app.py`, `app_helpers.py`, UI smoke tests	app starts, smoke tests green, uses real engine
`feat/docs-qa`	Ayush	README, docs, oracle, report, slides, screenshots	no code files touched; files render

- Team step (needs team approval, Aryan runs it): `git init`, create the GitHub repository, add collaborators, push the initial commit with this plan, create the three branches.
- Rebase or merge `main` into your branch at every checkpoint. Touch only files you own; if you need a change in someone else's file, message the owner.
- Tags: `mvp` (T+7:30), `v1.0-rc` (code freeze, T+14:30), `v1.0` (final, T+23:00).
- Ayush works through the shared repository; if Git is awkward, he shares files with Aryan who commits them, but the **authorship of each file stays with its owner**.

7. Work packages

Times are estimates of focused work. 'Waits' lists what must exist before the task can finish. Priority: **P0** = required for the MVP and for full marks; **P1** = strong enhancement after the MVP checkpoint; **P2** = optional, only when no P0 bug is open.

7.1 Aryan Rajguru: Game Theory engine and integration

Owns: `game_engine.py`, `sample_data.py`, engine and integration tests, `docs/MODEL.md`, `docs/INTERFACE.md`, `docs/results/`, `requirements.txt`, `.gitignore`, final integration and technical sign-off.
Must be able to explain: the whole model, the potential-game argument, every engine function, the test strategy.

ID	Pri	Deliverable and suggested path	Depends on	Acceptance criteria	Est.	Parallel with / waits for
A0	P0	`.gitignore`, `requirements.txt`; team step: `git init`, GitHub repo, branches (with team approval)	none	Fresh virtualenv: `pip install -r requirements.txt` succeeds; branches exist; plan files pushed	0.25h	Parallel with everything. Others wait at most 15 min for the repo
A1	P0	`docs/MODEL.md` v1: players, strategies, utility, parameters, tie-break, equilibrium definition, potential function and proof sketch, welfare/PoA/PoS/Pareto definitions, assumptions	none	Every symbol matches Section 4 and `INTERFACE.md`; Harsh and Ayush have read it; blocks Q4, Q5	0.75h	Parallel with F1, Q1-Q3
A2	P0	`docs/INTERFACE.md` and `game_engine.py` skeleton with all signatures, docstrings, exception classes (bodies `raise NotImplementedError`)	A1 draft	Matches Section 8 exactly; `import game_engine` works; pushed to `main`	0.5h	Harsh may keep using the mock from Section 8 until A4 lands
A3	P0	`sample_data.py`: `get_default_scenario()` (8 fictional students, skills, 3 projects, p, w, gamma) and `get_tiny_scenario()` (Section 8 example)	A1	Passes `validate_scenario`; w symmetric with zero diagonal; contains at least one start assignment with 2 or more profitable deviations; tiny scenario reproduces Section 8 numbers; data frozen at CP1	0.5h	Q4, Q5 wait for it
A4	P0	Engine core: `validate_scenario`, `validate_profile`, `crowding_cost`, `utility`, `utilities`, `social_welfare`, `potential`, `teams`, `best_response`, `profitable_deviations`, `is_nash`, error classes	A2, A3	Tests E-01..E-05, E-09, E-12 pass; inputs never mutated; output JSON-serialisable; dropped to `main` at T+3:30	1.5h	Harsh swaps mock for real functions at the drop; tests written alongside
A5	P0	`best_response_dynamics`, `enumerate_equilibria` (all equilibria, welfare optimum), generic `pure_nash_equilibria(strategy_counts, payoff_fn)`, `random_profile`, CLI `python game_engine.py --report`	A4	Every dynamics endpoint passes `is_nash` for 200 seeded starts; each step strictly raises Phi; equilibrium set non-empty on default data; runtime for 6,561 profiles measured and recorded	1.5h	Q7 and F6 wait for it
A6	P0	`tests/test_game_engine.py` covering E-01..E-14	A4, A5	`pytest -q tests/test_game_engine.py` green; each test states the source of its expected value (hand calculation or property, never 'whatever the engine printed')	1.5h	Parallel with F4-F7
A7	P0	`tests/test_integration.py` covering I-01..I-05	A5, F4	Green; includes invalid-input cases and schema checks against `INTERFACE.md`	0.75h	Waits for first UI helper drop

ID	Pri	Deliverable and suggested path	Depends on	Acceptance criteria	Est.	Parallel with / waits for
A8	P1	Extend `enumerate_equilibria` and `--report`: Pareto flag per equilibrium, welfare gaps, price of anarchy / stability (null when welfare is not strictly positive)	A5, CP3	Output equals Ayush's independent output; documented in `INTERFACE.md`	1h	F8 waits for it
A9	P0	Integration lead (T+7:30-10:30): review and merge branches, run full `pytest`, run the app, compare UI against CLI on 3+ assignments, tag `mvp`, triage bugs	A6, A7, F4-F7, Q7	CP3 and CP4 gate criteria met (Section 10)	3h	Runs while Ayush executes Q8
A10	P0	`docs/results/engine_report.json`, `pytest_output.txt`, `timing.txt` from the **frozen** build, with commit hash noted	CP5 freeze	Files regenerate with one documented command; numbers are the only source for the report	0.75h	Q10 and Q11 wait for it
A11	P1	Technical review of report (model, algorithm, results), slides, and technical answers in `docs/VIVA.md`	Q11, Q12	No claim without evidence in `docs/results/` or `TEST_LOG.md`; review comments resolved	1.5h	Parallel with F13
A12	P2	`--sweep-gamma`: equilibrium count and welfare against gamma	A8	Table printed; default behaviour unchanged; tests still green	1h	Only if no P0 bug is open

7.2 Harsh: frontend and interactive application

Owns: `app.py`, `app_helpers.py`, `tests/test_ui_smoke.py`. **Must be able to explain:** what each tab shows, how every number is obtained from the engine, the interface contract, and the limits of what 'stable' means.

ID	Pri	Deliverable and suggested path	Depends on	Acceptance criteria	Est.	Parallel with / waits for
F1	P0	`app.py` skeleton with tabs Scenario / Assignment / Equilibrium / Simulation, running on the Section 8 mock engine output	none	`streamlit run app.py` opens with no exception; command documented for README	0.5h	Parallel with A1-A3
F2	P0	Scenario tab: students and skills, projects, preference table, compatibility table (`st.dataframe`), gamma slider	F1, A3	Shows 8 students and 3 projects from `get_default_scenario()`; changing gamma recomputes dependent results	1h	Uses mock data until A3 (T+2h)
F3	P0	Assignment tab: one `selectbox` per student, 'Random (seed)' and 'Reset' buttons, state in `st.session_state`	F1, A2	Every profile built through the UI passes `validate_profile`; invalid state cannot be created	1h	Parallel with A4
F4	P0	Evaluation panel: teams by project, per-student utility table, welfare, potential, utility bar chart	A4	For 3 chosen assignments the numbers equal engine output and Ayush's oracle values	1.5h	Waits for A4 drop (T+3:30)
F5	P0	Equilibrium tab: calls `is_nash`; shows 'stable against unilateral deviations' or a table of deviations (student, from, to, gain) plus a plain-language sentence	A4	Non-equilibrium start lists exactly the engine's deviations; equilibrium shows none; wording never says 'optimal'	1.5h	Parallel with A5

ID	Pri	Deliverable and suggested path	Depends on	Acceptance criteria	Est.	Parallel with / waits for
F6	P0	Simulation tab: start profile, order option, run `best_response_dynamics`, trace table, step slider showing teams at each step, final badge computed by a fresh `is_nash` call, welfare/potential line chart	A5	Final state re-verified in the UI by `is_nash`, not by the `converged` flag; hitting `max_rounds` is displayed as 'not converged'	2h	Waits for A5 (T+5h)
F7	P0	`app_helpers.py` (display builders, `StratosError` boundary) and `tests/test_ui_smoke.py` using Streamlit `AppTest`	F1	S-01..S-04 pass: app loads; default run has no exception; simulation runs; invalid input shows an error, not a traceback	1h	Parallel with F6
F8	P1	Analysis tab: cached enumeration, list of all equilibria, optimum vs equilibrium welfare, price of anarchy / stability, Pareto flags, comparison chart	A8	Numbers equal CLI `--report`; text states 'stable is not the same as optimal'	1.5h	Waits for A8
F9	P0	Bug fixing from Q8 and A9 (two rounds: T+7:30-10:30 and T+13:30-14:30)	Q8	All P0/P1 bugs closed or documented in `TEST_LOG.md`	2h	Interleaved with F8
F10	P2	Demo-mode button loading the exact start assignment from `docs/DEMO_SCRIPT.md`	Q9	One click reproduces the demo start	0.75h	Optional
F11	P0	Freeze verification (T+14:30): fresh clone, install, run, full demo script three times; P0 fixes only	CP5	3 of 3 clean runs recorded in `TEST_LOG.md` by Ayush	1h	Parallel with A10, Q10
F12	P2	`st.data_editor` for preferences with validation; JSON/CSV download; theming	CP4, no open P0	No regression: pytest and smoke tests rerun green	1.5h	Optional; never after freeze
F13	P1	Review report implementation/UI sections and slides 6-8; rehearse the demo	Q11, Q12	Review comments resolved; demo timed under 7 minutes	1h	Parallel with A11

7.3 Ayush: quality assurance, documentation and presentation

Owns: README, test plan and log, oracle checks, report, slides, demo script, viva file, screenshots. Roughly 40% of the work is verification (Q4, Q5, Q7, Q8, Q15) and executed against the real code and app, not documents. He uses ChatGPT for the independent implementation: paste only `docs/MODEL.md` and `sample_data.py` data, **never** `game_engine.py`. **Must be able to explain:** how each result was independently verified and what the evidence is.

ID	Pri	Deliverable and suggested path	Depends on	Acceptance criteria	Est.	Parallel with / waits for
Q1	P0	`docs/ASSIGNMENT_CHECK.md`: exact title and group row from `Groups.xlsx`, handout topics, submission format and any rules	`Groups.xlsx`, handout	Team informed by T+0:30; any mismatch with Assumption A-1/A-2/A-3 flagged	0.5h	Parallel with A0-A2
Q2	P0	`README.md` skeleton: overview, requirements, setup (venv, install), run, test, repo map, team	A0	Commands tested from a clean clone at T+7:30; final pass at T+22h	1h	Parallel
Q3	P0	`docs/TEST_PLAN.md` from Section 11: IDs, objective, input, source of expected value, owner, status 'Planned'	this plan	Every test topic in the brief maps to at least one ID	1h	Parallel

ID	Pri	Deliverable and suggested path	Depends on	Acceptance criteria	Est.	Parallel with / waits for
Q4	P0	`tests/oracle/independent_check.py` and `independent_output.txt`: written with ChatGPT from `MODEL.md` only; imports data from `sample_data.py`; enumerates all 3^8 profiles; prints all pure equilibria, welfare optimum, welfare of each equilibrium	A1, A3	Runs locally; **does not import** `game_engine`; output saved; Aryan reviews the code for conformity to `MODEL.md`	1.5h	Starts T+1h; finishes T+2:30
Q5	P0	`tests/oracle/oracle_check.xlsx`: Excel formulas computing utilities and potential for the 4 profiles of the tiny example and 3 default-scenario profiles	A1, A3	Values come from formulas, not typed numbers; tiny example matches Section 8	1h	Parallel with A4
Q6	P0	`docs/REPORT.md` v0: cover, abstract draft, problem, objectives, background, model, architecture, applications, limitations, references (only sources actually read); results sections marked `[TO FILL FROM ACTUAL RUN]`	A1, A2, F1	No invented result; every placeholder visible; Aryan reads model section	2.5h	Spread over T+3:30-7:30
Q6b	P0	`presentation/SLIDES.md`: 12-slide text skeleton (title, 3-5 bullets, planned visual, presenter)	Q6	Follows Section 12 story order	0.75h	T+10:30-12:30
Q7	P0	Cross-check engine vs independent checker vs Excel oracle; record O-01..O-04 in `docs/TEST_LOG.md`	A5, Q4, Q5	Equilibrium sets identical, or a discrepancy report (profile, both values) sent to Aryan within 15 minutes	1.5h	T+5h-6:30; blocks CP3
Q8	P0	Manual UI test round 1 on the real app from a clean clone: checklist M-01..M-12; bug reports with steps, expected, actual	CP3	Each case Pass/Fail with date and commit hash; failures sent to Harsh/Aryan	2.5h	Runs while Aryan integrates
Q9	P1	`docs/DEMO_SCRIPT.md`: 6-8 steps with exact clicks, spoken points, **expected values copied from the actual run**	CP4	Rehearsed once end to end; reviewed by Aryan and Harsh	1h	T+10:30-12:30
Q10	P0	Screenshots S1-S8 in `screenshots/` plus `INDEX.md` (file, tab, state, commit hash)	CP5 freeze	All from the frozen build; none edited or mocked	1.5h	T+14:30-16:00
Q11	P0	Final `docs/REPORT.md` and exports `docs/STRATOS_Report.docx` / `.pdf`; results and testing sections from `docs/results/` and `TEST_LOG.md`	A10, Q10	Zero `[TO FILL]` markers; every number traceable to a results file or log line	2h	T+16:00-17:30
Q12	P0	`presentation/STRATOS.pptx`, 12 slides, built in PowerPoint from `SLIDES.md` with real screenshots	Q10, Q6b	Numbers equal report; text readable; each member presents 3-4 slides	1.5h	T+17:30-19:00
Q13	P0	`docs/VIVA.md`: at least 20 questions with answers, technical ones reviewed by Aryan	A11	Each member can answer their own part unaided in rehearsal	1h	T+19:00-20:00
Q14	P0	Final consistency check and clean-clone run; table in `TEST_LOG.md` comparing code, test log, report, slides	CP7	Every number identical across results file, report and slides; README commands work	1.5h	T+20:30-22:00
Q15	P2	Post-freeze regression round and random-start statistics cross-check	CP5	Logged in `TEST_LOG.md`; no code change by Ayush	1h	Optional

8. Dependencies and interface contract (engine <-> app)

The contract below is frozen at checkpoint CP1 (T+2h). Any later change requires all three members' agreement and an update of `docs/INTERFACE.md`; the engine exposes `INTERFACE_VERSION = "1.0"`.

8.1 Dependency map

Task	Blocked until	Mitigation while waiting
F2-F6 (real data and engine)	A2 skeleton (T+1:30), A3 data (T+2h), A4 (T+3:30), A5 (T+5h)	Harsh builds on the mock outputs in 8.4, which are valid under the contract
Q4, Q5 (independent checks)	A1 `MODEL.md` (T+1h) and A3 data (T+2h)	Ayush starts with the tiny 2x2 example in 8.4 whose answers are hand-derived
Q7 (cross-check)	A5 CLI `--report`, Q4, Q5	Compare the tiny example first
Q8 (manual UI testing)	CP3 tag `mvp`	Prepare checklist and clean-clone setup before CP3
Q10-Q12 (screenshots, report results, slides)	CP5 freeze and A10 results	Draft text with visible placeholders only; **no numbers before the freeze**

8.2 Input schemas (JSON-serialisable)

```
scenario = {
  "students": ["Asha", ...],            # list[str], n >= 1, unique
  "projects": ["Web Portal", ...],      # list[str], m >= 2, unique
  "pref":     [[5, 1], [1, 5], ...],    # n x m numbers (integers recommended), 0..10 suggested
  "w":        [[0, 2], [2, 0], ...],    # n x n numbers, symmetric, zero diagonal
  "gamma":    1,                        # number >= 0;   c(k) = gamma * (k - 1)**2
  "skills":   {"Asha": ["Design"], ...}   # optional metadata for display; ignored by the engine
}
profile = [0, 1, 2, ...]                 # list[int], len n, each in 0..m-1 (0-based project index)
```

8.3 Functions (module `game_engine`)

Function	Returns	Notes
`validate_scenario(scenario)`	None	Raises `ScenarioError` (missing key, ragged or non-numeric matrix, asymmetric w, non-zero diagonal, negative gamma, duplicate or empty names, bool values)
`validate_profile(profile, scenario)`	None	Raises `ProfileError` (wrong length, non-int, out-of-range index)
`crowding_cost(k, scenario)`	number	`gamma * (k-1)**2`
`utility(i, profile, scenario)`	number	u_i(s)
`utilities(profile, scenario)`	list[number]	u_i for all i
`social_welfare(profile, scenario)`	number	sum of utilities
`potential(profile, scenario)`	number	Phi(s), for tests and trace charts
`teams(profile, scenario)`	list[list[int]]	student indices on each project, ascending
`best_response(i, profile, scenario)`	dict	`{student, current, best, improves, options:[{project, utility}]}`
`profitable_deviations(profile, scenario)`	list[dict]	each `{student, from_project, to_project, current_utility, new_utility, gain}`, sorted by student then project; only gain > EPS
`is_nash(profile, scenario)`	dict	`{is_nash, utilities, deviations}`; validates inputs
`best_response_dynamics(scenario, start, order="round_robin", seed=None, max_rounds=100)`	dict	`{converged, rounds, start_profile, final_profile, steps, is_nash_final}`; `order` is `round_robin` or `random` (seeded permutation each sweep); `rounds` counts sweeps including the final no-change sweep

Function	Returns	Notes
`enumerate_equilibria(scenario, with_analysis=True)`	dict	`{n_profiles, equilibria:[{profile, utilities, welfare, potential, pareto_optimal}], optimal:{profile, welfare}, welfare_gap_best_equilibrium, welfare_gap_worst_equilibrium, price_of_anarchy, price_of_stability}`; ratios are null unless all welfare values are strictly positive; `equilibria` may be an empty list
`pure_nash_equilibria(strategy_counts, payoff_fn)`	list[list[int]]	generic checker used for the no-equilibrium test game
`random_profile(scenario, seed)`	list[int]	seeded random assignment
CLI `python game_engine.py --report`	JSON on stdout	default scenario: equilibria, optimum, timing; used as report evidence

8.4 Error behaviour and a hand-verified example

- Engine functions never print, never mutate arguments, never return NumPy types or tuples, and are deterministic given `seed`.
- `StratosError` (subclass of `ValueError`) is the base of `ScenarioError` and `ProfileError`; messages name the offending field and index.
- The UI catches `StratosError`, shows `st.error(str(exc))`, and keeps its previous valid state.

Tiny example (two students, two projects, gamma = 1, so c(1) = 0 and c(2) = 1). Arithmetic below was derived by hand and re-checked with a throw-away script during planning; it is a **test fixture to be re-verified by A6**, not a project result.

```
pref = [[5, 1], [1, 5]]      w = [[0, 2], [2, 0]]      gamma = 1
profile  utilities  welfare  Phi   pure Nash?
[0, 0]   [6, 2]       8       7    no  (student 1 gains 3 by moving to project 1)
[0, 1]   [5, 5]      10      10    yes (unique equilibrium, also the welfare optimum)
[1, 0]   [1, 1]       2       2    no  (students 0 and 1 each gain 5 by moving to the other project)
[1, 1]   [2, 6]       8       7    no  (student 0 gains 3 by moving to project 0)
Potential check: Phi([0,1]) - Phi([0,0]) = 3 = u_1([0,1]) - u_1([0,0]) = 5 - 2
```

```
from sample_data import get_tiny_scenario
from game_engine import is_nash, best_response_dynamics, enumerate_equilibria
sc = get_tiny_scenario()
is_nash([0, 0], sc)
# expected shape: {"is_nash": False, "utilities": [6, 2], "deviations": [{"student": 1, "from_project": 0,
#   "to_project": 1, "current_utility": 2, "new_utility": 5, "gain": 3}]}
best_response_dynamics(sc, [0, 0])
# expected: converged True, rounds 2, final_profile [0, 1], one step (student 1: 0 -> 1, gain 3), is_nash_final True
enumerate_equilibria(sc)
# expected: n_profiles 4, equilibria [[0, 1]], optimal [0, 1] with welfare 10, welfare gaps 0, PoA = PoS = 1.0
```

9. Hour-by-hour execution plan

T+0 is the moment all three members start. Task IDs refer to Section 7. If everyone is available the whole day, the 1-hour break at T+12:30 can be shortened, but **do not remove it**: the code freeze at T+14:30 needs clear heads.

Time	Aryan	Harsh	Ayush	Checkpoint / gate
T+0:00-1:00	A0 repo and requirements. A1 `MODEL.md` v1	F1 environment and app skeleton on mock output	Q1 check `Groups.xlsx` and handout. Q2 README skeleton. Start Q3	CP0 at T+0:30: all read this plan; Ayush reports assignment facts; repo exists
T+1:00-2:00	A2 `INTERFACE.md` + skeleton. A3 `sample_data.py`	F2 scenario tab (mock data), then A3 data	Finish Q3. Start Q4 (paste `MODEL.md` into ChatGPT)	**CP1 at T+2:00: MODEL.md, INTERFACE.md, skeleton and sample data frozen on** `main`**.** Aryan presents 5 minutes; changes need all three
T+2:00-3:30	A4 engine core, tests alongside (E-01..E-05, E-09, E-12)	F3 assignment tab; start F4	Finish Q4. Start Q5 Excel oracle	A4 dropped on `main` at T+3:30
T+3:30-5:00	A5 dynamics, enumeration, CLI	Switch to real engine; F4, F5	Finish Q5. Start Q6 report v0	**CP2 at T+5:00:** engine functions complete, `--report` runs, core tests green
T+5:00-6:30	A6 remaining tests (E-06..E-14)	F6 simulation tab	Q7 cross-check (O-01..O-04); continue Q6	Discrepancy rule: report to Aryan within 15 minutes; stop and hand-compute
T+6:30-7:30	A7 integration tests; prepare merge	F7 helpers, error boundary, smoke tests	Q2 clean-clone run of README; finish Q6	**CP3 at T+7:30: MVP.** App runs end to end on real engine, `pytest` green, tag `mvp`
T+7:30-10:30	A9 integration lead: review, merge, UI-vs-CLI comparison, triage	F9 fix bugs as they arrive; sketch F8 layout on mock output	Q8 manual UI test round 1 (M-01..M-12), bug reports	**CP4 at T+10:30:** zero open P0 bugs, else P1/P2 cancelled and bug fixing continues
T+10:30-12:30	A8 Pareto, price of anarchy/stability (P1); review `DEMO_SCRIPT`	F8 analysis tab (P1)	Q9 `DEMO_SCRIPT.md`. Q6b `SLIDES.md` skeleton	Only P1 work; no refactoring; no new dependencies
T+12:30-13:30	Break	Break	Break	Staggered if needed; no merges during the break
T+13:30-14:30	Final review, full test run	F9 second bug round; F10 only if idle	Re-test fixed bugs (regression)	**CP5 at T+14:30: CODE FREEZE, tag** `v1.0-rc`**.** After this only P0 fixes, with all three agreeing
T+14:30-16:00	A10 results snapshot from frozen build	F11 three clean demo runs from fresh clone	Q10 screenshots S1-S8 and `INDEX.md`	Evidence capture; nobody edits code
T+16:00-17:30	A11 review of model/algorithm text	F13 review UI and implementation text	Q11 report results and testing sections, export `.docx/.pdf`	**CP6 at T+17:30:** report v1 with real screenshots and results, no `[TO FILL]` left
T+17:30-19:00	Review report math; check slides 4, 5, 9	Check slides 6-8; prepare demo machine	Q12 build `STRATOS.pptx`	Slides use only numbers from `docs/results/`
T+19:00-20:00	Write technical viva answers; review `VIVA.md`	Review demo-related answers	Q13 `VIVA.md`	**CP7 at T+20:00:** report, slides, viva file complete
T+20:00-22:00	Rehearsal 1 and 2 (timed); clean-clone run on a second machine if possible	Rehearsal 1 and 2; record backup screen recording	Q14 consistency check; update README	**CP8 at T+22:00:** two timed rehearsals done; every member answered cross-questions on the others' parts

Time	Aryan	Harsh	Ayush	Checkpoint / gate
T+22:00-23:00	Final tag `v1.0`, package files	Final demo-machine check	Final folder check against Section 14 checklist	**CP9 at T+23:00:** deliverables packaged and tagged
T+23:00-24:00	Buffer / submission	Buffer / submission	Buffer / submission	Do not start new work

9.1 Parallel work and forced waits

Can run fully in parallel	Must wait for another member
T+0-2h: A0-A3, F1-F2, Q1-Q3 (nobody blocks anybody). T+2-7:30: engine build, UI build, independent check and report draft run side by side. T+14:30-17:30: A10, F11, Q10, Q11 run side by side once the freeze exists.	F4-F6 need A4/A5 drops. Q4/Q5 need `MODEL.md` and `sample_data.py`. Q7 needs A5 and Q4/Q5. Q8 needs the `mvp` tag. Q10-Q12 need the freeze and A10. Nothing in the report may contain a result number before A10.

10. Priorities and acceptance criteria

10.1 Checkpoint gates

Gate	Time	Pass criteria (all must hold)	If failed
CP0	T+0:30	Plan read; repo plan agreed; Ayush's assignment findings reported	Resolve now; do not start coding on a wrong topic
CP1	T+2:00	`MODEL.md`, `INTERFACE.md`, engine skeleton, `sample_data.py` on `main`; Harsh runs the app on mock output; Ayush has the independent checker underway	Aryan delivers by T+2:30 maximum; Harsh continues on mock
CP2	T+5:00	A4 and A5 done; `python game_engine.py --report` prints equilibria; core tests green; Harsh has tabs F2-F5 on the real engine	Drop A8 to P2; Ayush tests the tiny example only
CP3	T+7:30	End-to-end MVP: select assignment, see utilities, equilibrium check, run dynamics, see verified final state; `pytest` green; tag `mvp`; independent output and oracle available	Fallback F-B (Section 14); P1 cancelled
CP4	T+10:30	Q8 round 1 finished; zero open P0 bugs; UI numbers equal CLI and oracle on 3+ assignments	P1/P2 cancelled; two extra hours of bug fixing
CP5	T+14:30	Code freeze: `pytest` green, smoke tests green, three clean demo runs, tag `v1.0-rc`	No new features; only agreed P0 fixes
CP6	T+17:30	Report v1 complete with actual screenshots and numbers from `docs/results/`	Cut P1 report sections; keep required ones
CP7	T+20:00	Slides (12), `VIVA.md`, `DEMO_SCRIPT.md` complete	Reduce slides to 10; keep story order
CP8	T+22:00	Two timed rehearsals; consistency table filled; README works from a clean clone	Fix README/consistency first, rehearsal second
CP9	T+23:00	Files packaged, `v1.0` tagged, backups saved	Submit what is verified

10.2 Definition of done

Level	Meaning
P0 done	Implemented, tests pass, reviewed by one other member, evidence recorded in the test log
P1 done	As P0, and demonstrated in the live demo script; if unreliable, removed rather than shown
P2 done	Only counts if no regression; never started after code freeze

Rule: do not let report writing or UI polish delay mathematical correctness or end-to-end functionality. A feature that is not verified is not shown.

11. Test plan

> **Status of all tests below: PLANNED. Nothing has been executed.** Executed results, with date, commit hash and actual output, will be recorded only in `docs/TEST_LOG.md`. Expected values come from hand calculation, mathematical properties or the independent implementation, never from copying engine output.

11.1 Automated engine tests (`tests/test_game_engine.py`, owner Aryan)

ID	Topic	Input and expected-value source	Status
E-01	Utility calculation	Tiny example: utilities of all 4 profiles equal the hand-derived table in Section 8	Planned
E-02	Profile validity	Wrong length, non-int, negative index, index >= m raise `ProfileError`	Planned
E-03	Scenario validity	Asymmetric w, non-zero diagonal, ragged matrix, negative gamma, duplicate names, bool or string values raise `ScenarioError`	Planned
E-04	Known pure Nash equilibrium	Tiny example: [0, 1] is the unique equilibrium (hand-derived); `is_nash` true with no deviations	Planned

ID	Topic	Input and expected-value source	Status
E-05	Profitable unilateral deviation	Tiny example: [0, 0] has exactly one deviation (student 1 to project 1, gain 3); [1, 0] has two deviations of gain 5	Planned
E-06	No pure equilibrium handled honestly	Matching pennies through `pure_nash_equilibria`: returns an empty list; the result states there is none	Planned
E-07	Dynamics end in equilibrium	200 seeded random starts on the default scenario: final profile passes `is_nash`; every step has gain > 0 and strictly raises Phi	Planned
E-08	Enumeration consistency	Every enumerated equilibrium passes `is_nash`; every E-07 endpoint is in the enumerated set; set equals Ayush's independent output (O-01)	Planned
E-09	Team-size / crowding effect	With w = 0: adding a student to a team of size k changes each existing member's utility by exactly c(k) - c(k+1)	Planned
E-10	Constraint behaviour	Very large gamma (for example 1000): the all-in-one-team profile is not an equilibrium	Planned
E-11	Determinism and tie-breaking	Same seed gives identical trace; ties go to the lowest project index; inputs unchanged after calls	Planned
E-12	Exact potential identity	At least 1,000 random (profile, student, new project) triples: delta u equals delta Phi; welfare equals sum of utilities	Planned
E-13	Non-convergence reported	`max_rounds=1` on a start needing more sweeps: `converged` false and not claimed stable	Planned
E-14	Structural invariants	Team sizes always sum to n; empty teams allowed; utility of a lone student equals preference minus c(1)	Planned

11.2 Integration, automated UI smoke and cross-check tests

ID	Topic	Input and expected-value source	Owner	Status
I-01	Outputs JSON-serialisable	`json.dumps` on every public function result	Aryan	Planned
I-02	Schema conformity	Keys and types equal `INTERFACE.md`	Aryan	Planned
I-03	Error behaviour	Invalid inputs raise `StratosError` subclasses with informative messages	Aryan	Planned
I-04	Engine-to-UI helpers	`app_helpers` builders accept real engine output without exceptions	Aryan	Planned
I-05	CLI report	`--report` JSON parses and equals `enumerate_equilibria` on the default scenario	Aryan	Planned
S-01..S-04	App loads; default run has no exception; simulation runs; bad input yields error text	Streamlit `AppTest`	Harsh	Planned
O-01	Equilibrium set equality	Independent checker vs engine on the default scenario	Ayush	Planned
O-02	Welfare optimum equality	Same, optimum profile and value	Ayush	Planned
O-03	Excel oracle	Utilities of 3 default profiles equal engine and UI values	Ayush	Planned
O-04	Tiny example	Excel, independent script and engine all match Section 8	Ayush	Planned

11.3 Manual UI checklist (M-01..M-12, executed by Ayush from a clean clone)

ID	Check	Pass condition
M-01	Setup from README	Fresh venv, `pip install -r requirements.txt`, `streamlit run app.py` works with only the README commands
M-02	Scenario display	8 students and 3 projects visible; values equal `sample_data.py`
M-03	Assignment entry	Every student can be set to every project; profile shown matches selections
M-04	Displayed vs computed utilities	For 3 assignments, UI utilities equal Excel oracle (O-03)
M-05	Non-equilibrium detection	Deviation table equals the independent calculation
M-06	Equilibrium recognition	An enumerated equilibrium is reported stable with no deviations
M-07	Best-response simulation	Trace steps each show a positive gain; final state flagged stable and re-verified

ID	Check	Pass condition
M-08	Gamma slider	Changing gamma changes results consistently with the CLI for the same gamma
M-09	Invalid and edge input	Extreme gamma, repeated clicks, reset mid-run: error message, no traceback, no stale results
M-10	Analysis tab (P1)	Equilibrium list and welfare values equal `--report`
M-11	Refresh and re-run	Page refresh returns to a defined state; no crash
M-12	Wording audit	No claim that an equilibrium is 'optimal' or 'best'

Recording rules: each executed test gets date, tester, commit hash, input, actual output, Pass/Fail. Failures are logged as bugs (steps, expected, actual) and retested after the fix.

12. Report and presentation outlines

12.1 Report outline (`docs/REPORT.md` to `.docx` / `.pdf`)

#	Section	Content	Writes	Evidence source
0	Cover page and team details	Title, course CSE3145, institution, members, date, group per `Groups.xlsx`	Ayush	`ASSIGNMENT_CHECK.md`
1	Abstract	Problem, model, method, result in about 150 words	Ayush	Written last, from actual results
2	Problem statement	Strategic behaviour in team formation, why individual incentives differ from group welfare	Ayush	Brief
3	Objectives	Model, simulate, verify equilibria, compare with welfare optimum, tested application	Ayush	Section 3 of this plan
4	Background	Strategies, payoffs, best response, Nash equilibrium, potential games, Pareto optimality, welfare	Ayush (Aryan reviews)	Course handout (if found), textbooks actually consulted
5	Mathematical model and assumptions	Players, strategies, utility, parameters, equilibrium condition, potential argument, synthetic-data disclosure	Ayush (Aryan reviews)	`MODEL.md`
6	System architecture	Diagram, data flow, interface contract	Ayush (Harsh reviews)	Section 5, `INTERFACE.md`
7	Methodology and algorithm	Best-response dynamics, enumeration, verification, complexity	Ayush (Aryan reviews)	`MODEL.md`, engine code
8	Implementation and technology stack	Modules, libraries, how to run	Ayush (Harsh and Aryan review)	Repository, `README.md`
9	Testing and results	Executed tests, equilibrium list, welfare comparison, timings, cross-checks	Ayush	`TEST_LOG.md`, `docs/results/` (actual only)
10	Screenshots	S1-S8 with captions and commit hash	Ayush	`screenshots/INDEX.md`
11	Applications	Course project allocation, lab slots, hackathon teams, congestion-style resource allocation	Ayush	Reasoned discussion, no invented case studies
12	Limitations and future work	Synthetic data, assumed weights, complete information, unilateral deviations only, small n; correlated equilibrium, mechanism design / VCG, repeated play	Ayush (Aryan reviews)	Honesty rules in Section 4
13	Conclusion	What was shown, what was not	Ayush	Results
14	References	Only sources actually consulted (textbook, handout, Streamlit docs)	Ayush	Verified by the author

12.2 Presentation outline (12 slides, about 10 minutes plus demo)

Slide	Title	Content and visual	Presenter
1	STRATOS: title and team	Title, course, members, one-line problem	Ayush
2	The problem	Students choose teams strategically; individual vs group benefit; example	Ayush
3	Game Theory concepts used	Strategies, payoffs, best response, Nash equilibrium, potential, welfare, Pareto	Ayush
4	The model	Players, strategies, utility formula with a worked tiny example	Aryan
5	Algorithm and guarantees	Best-response dynamics, potential argument, verification, complexity	Aryan
6	System architecture	Diagram from Section 5, module ownership	Harsh
7	The application	Screenshots S1-S4: scenario, assignment, evaluation, equilibrium check	Harsh
8	Live demo	Script from `DEMO_SCRIPT.md` (about 5 minutes); backup screenshots	Harsh with Aryan
9	Results	Actual equilibrium list, welfare comparison, convergence behaviour (numbers from `docs/results/`)	Aryan
10	Verification and testing	Four-way agreement: engine, independent code, Excel oracle, UI; executed test counts from `TEST_LOG.md`	Ayush
11	Applications and limitations	Where this applies; what the model does not capture	Harsh
12	Conclusion and questions	Key takeaways: stable is not optimal; what we would do next	Ayush

13. Demo plan and viva preparation

13.1 Reproducible live demo (about 6 minutes)

Fixed scenario, fixed start assignment, fixed seed; commands: `pip install -r requirements.txt`, `streamlit run app.py`, `pytest -q`. Expected values are **to be copied from the actual run** into `docs/DEMO_SCRIPT.md` after CP3; none are written here.

Step	Action	What to say / point out	Presenter
1	Scenario tab	Eight fictional students, three projects, preferences, skills; payoff = preference + teammate compatibility - crowding	Harsh
2	Assignment tab: load the prepared unstable start	Teams, each student's utility, social welfare	Harsh
3	Equilibrium tab	Not stable: list of profitable unilateral deviations (student, from, to, gain)	Aryan
4	Simulation tab: run best-response dynamics, step through the trace	Each move raises the mover's utility and the potential; stops when nobody can gain	Aryan
5	Equilibrium tab on the final state	Fresh `is_nash` check says stable; this means no single student gains by switching, nothing more	Aryan
6	Analysis tab (P1)	All pure equilibria found among 6,561 assignments; compare welfare with the optimum; stable is not optimal	Harsh
7	Change gamma and rerun	Stronger crowding penalty changes the equilibrium and team sizes	Harsh
8	Terminal: `pytest -q` and independent checker output	Tests and an independent implementation agree with the engine	Ayush

Backups: screenshots S1-S8 in the slides, a recorded screen capture of the full demo (made at T+20-22h), and the CLI `--report` output.

13.2 Likely viva questions and accurate answers

Final answers go into `docs/VIVA.md`; every member should be able to answer all of them.

Question	Answer
What game did you model?	A static, simultaneous-move, complete-information game. Players are students, each strategy is a choice of project, payoff is preference plus teammate compatibility minus a crowding cost.
Why game theory and not optimisation?	Students pursue their own payoff, not total welfare. The questions are which outcomes are stable against individual switching and how far they are from the best collective outcome.
Define a Nash equilibrium here.	An assignment where no student can raise their own utility by switching to another project while everyone else stays put.
How do you check it?	`is_nash` evaluates all 8 x 2 = 16 unilateral deviations for the profile and reports every one with positive gain.
Why does a pure equilibrium exist in your model?	The game is an exact potential game: a unilateral move changes the mover's utility by exactly the change in Phi. This needs symmetric compatibility and a crowding cost depending only on team size. A maximiser of Phi is an equilibrium.
Does best-response dynamics always converge?	In this model yes, because each strict improvement raises Phi and there are finitely many profiles. In general games it can cycle; matching pennies has no pure equilibrium, which is why we test it. The code still checks the final profile.
Is the equilibrium unique, and is it optimal?	Not necessarily either. The enumeration lists all pure equilibria. We report the welfare optimum separately, plus the welfare gap and a Pareto flag. Stable does not mean best.
What are price of anarchy and price of stability?	Optimum welfare divided by the welfare of the worst or best equilibrium, respectively; we only report the ratios when all welfare values are strictly positive and always report the gap.
Why no mixed strategies?	A pure equilibrium exists, so none is needed. Mixed strategies matter when none exists, as in matching pennies, which we use as a test.
Is this a dynamic game?	No. The game is static. Best-response dynamics is an adjustment process we simulate, not a sequential extensive-form game. Repeated games are not modelled.
How are ties handled?	A student moves only for a strictly positive gain, and the best project is the lowest-index one among ties. The equilibrium definition is weak: no strictly profitable deviation.
Why a soft crowding cost instead of a team-size cap?	A hard cap makes some deviations infeasible and changes the equilibrium definition; a convex cost keeps the potential-game structure and still discourages crowding.
What if compatibility were not symmetric?	The potential argument would no longer apply, so a pure equilibrium is not guaranteed. The code would still check and report honestly.
What is the complexity?	One sweep of dynamics is O(n^2 m); exhaustive enumeration is O(m^n n^2 m), about 1.3 million basic operations here, exponential in n, so the enumeration only works for small classes while the dynamics scales.
How do you know the code is correct?	Hand-computed examples, property tests (potential identity, determinism), a no-equilibrium test game, an independent re-implementation, an Excel oracle, and UI-versus-CLI comparison. Executed results are in `TEST_LOG.md`.
Where does the data come from?	It is fictional and synthetic, and the weights are modelling assumptions chosen to make strategic behaviour visible. We do not claim it measures real students.
What are the main limitations?	Synthetic data, assumed additive pairwise payoffs, complete information, only unilateral deviations (coalitions can do better), small class size, no communication or bargaining.
What would you add with more time?	Coalition-proof analysis, mechanism design such as VCG or stable matching to avoid strategic problems, repeated interaction, estimating payoffs from real surveys.
Real-world applications?	Course project allocation, lab or tutorial slot choice, hackathon teams, and other congestion-style resource-allocation problems.
What did each of you do?	Aryan: model, engine, tests, integration. Harsh: application, UI tests, demo. Ayush: independent verification, test log, report, slides, viva. Each of us can explain the others' parts.

14. Risks, fallback strategy and final checklist

14.1 Risks and mitigations

Risk	Likelihood / impact	Mitigation
Assigned title or rules in `Groups.xlsx` differ from the brief	Low / high	Q1 in the first 30 minutes; adjust scope at CP0
Interface drift between engine and app	Medium / high	Freeze at CP1; `INTERFACE.md` with version; integration tests I-01..I-05
Engine bug that nobody notices	Medium / very high	Four independent checks (hand examples, property tests, independent script, Excel); discrepancy stops all other work
Synthetic data gives an uninteresting result (for example everybody on one project)	Medium / medium	Tune `sample_data.py` **before CP1 only**, and disclose in the report that data and weights are synthetic; after the freeze report whatever is true
Streamlit version differences on teammates' machines	Medium / medium	Minimum versions in `requirements.txt`; Ayush's clean-clone run at T+7:30
Ayush cannot run Python locally	Low / medium	ChatGPT code interpreter for Q4; Excel for Q5; Q8 as a screen-share session driven by Harsh; Harsh captures screenshots, Ayush logs and verifies
Merge conflicts	Medium / low	One owner per file; branches per member; Aryan integrates
Unexplainable AI-generated code	Medium / high (viva)	Every owner reads and can explain their code; cross-explanations in rehearsal; keep functions small
Report or slides contain numbers that do not match the app	Medium / high	Numbers only from `docs/results/` after freeze; Q14 consistency table
Demo machine or network fails	Low / high	Backup screenshots and screen recording; CLI `--report`; works offline
Floating-point phantom deviations	Low / medium	Integer inputs, EPS = 1e-9 comparisons, tests E-11/E-12
Documentation time crowding out testing	Medium / high	Ayush's schedule alternates verification and writing; no result text before freeze

14.2 Fallback strategy if the team falls behind

Trigger	Action
Engine later than T+6:00	Cancel A8 (Pareto, price of anarchy/stability) and F8; show equilibrium list from `--report` only
F-B: UI unstable at CP3	Reduce to three tabs (Assignment, Equilibrium, Simulation), no charts; Aryan adds `python game_engine.py --demo` as a console demo; keep tests
No time for the Analysis tab	Demonstrate the enumeration through the CLI `--report` and a screenshot of its output
Report behind at CP6	Write the required sections only; use CLI output and screenshots as evidence; cut applications detail
Slides behind at CP7	Use 10 slides (merge 10 into 9, merge 11 into 12); keep the story order
Engine and independent checker disagree	Stop everything else; hand-compute the profile; fix the code or the checker; record the incident in `TEST_LOG.md`
Live app fails during the viva	Switch to the recorded demo and CLI output; explain from the screenshots

14.3 Final checklist

Done	Item	Owner
[ ]	`pip install -r requirements.txt` and `streamlit run app.py` work from a clean clone using only the README	Ayush verifies
[ ]	`pytest` passes; output saved in `docs/results/pytest_output.txt`	Aryan
[ ]	Engine, independent script, Excel oracle and UI agree; recorded in `TEST_LOG.md`	Ayush
[ ]	No claim that an equilibrium is 'optimal'; no unverified numbers anywhere	All

Done	Item	Owner
[ ]	Screenshots S1-S8 are from the frozen build and indexed	Ayush
[ ]	Report exported as `.docx` and `.pdf`; no `[TO FILL]` markers; every number traceable	Ayush
[ ]	`STRATOS.pptx` has 12 slides, numbers equal the report, backup screenshots included	Ayush
[ ]	`DEMO_SCRIPT.md` rehearsed twice with timing under 7 minutes; backup screen recording saved	Harsh and Aryan
[ ]	`VIVA.md` answered unaided by each member on every section	All
[ ]	Synthetic-data disclosure present in report, slides and README	Ayush
[ ]	Repository tagged `v1.0`; all files committed by their owners	Aryan

14.4 First actions for the next 30 minutes

1. **All:** read Sections 4, 7 and 8; raise objections now, not at T+5h.
2. **Aryan:** approve repo creation with the team, then A0, A1, A2.
3. **Harsh:** `python --version`, `pip install streamlit pandas pytest`, start F1 on the Section 8.4 example.
4. **Ayush:** open `Groups.xlsx` and the handout, complete Q1, then Q2 and Q3; tell the team whether any assumption in Section 1.2 is wrong.